

Algoritmos de Ordenação — Bubble Sort e Shell Sort

Seminário – Estruturas de Dados | Equipe 3 | Turma B

Bubble Sort

O que é

Bubble Sort é um algoritmo de ordenação que percorre o array repetidamente, comparando elementos **dois a dois** e trocando-os de posição quando estão fora de ordem.

A cada passagem completa, o **maior elemento ainda não ordenado "borbulha"** para o final do array — daí o nome.

Como funciona

1. Começa no índice 0 e compara `array[j]` com `array[j+1]`
2. Se `array[j] > array[j+1]`, troca os dois
3. Avança um índice e repete
4. Ao fim de cada passagem, o maior elemento está na posição correta
5. Reduz o intervalo de comparação e repete tudo até não haver mais trocas

Exemplo passo a passo

Array inicial: [64, 34, 25, 12, 22, 11, 90]

Passagem	Array resultante
1	[34, 25, 12, 22, 11, 64, 90]
2	[25, 12, 22, 11, 34, 64, 90]
3	[12, 22, 11, 25, 34, 64, 90]
4	[12, 11, 22, 25, 34, 64, 90]
5	[11, 12, 22, 25, 34, 64, 90]
6	[11, 12, 22, 25, 34, 64, 90]

Array final: [11, 12, 22, 25, 34, 64, 90] 

Código Java

```
public class BubbleSort {
```

```

public static void bubbleSort(int[] array) {
    int n = array.length;

    for (int i = 0; i < n - 1; i++) {
        // A cada passagem, o maior elemento "borbulha" para o final
        for (int j = 0; j < n - 1 - i; j++) {
            if (array[j] > array[j + 1]) {
                // Troca os elementos
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
            }
        }
        System.out.print("Passagem " + (i + 1) + ": ");
        imprimirArray(array);
    }
}

public static void imprimirArray(int[] array) {
    System.out.print("[");
    for (int i = 0; i < array.length; i++) {
        System.out.print(array[i]);
        if (i < array.length - 1) System.out.print(", ");
    }
    System.out.println("]");
}

public static void main(String[] args) {
    int[] array = {64, 34, 25, 12, 22, 11, 90};

    System.out.println("Array original:");
    imprimirArray(array);
    System.out.println();

    bubbleSort(array);

    System.out.println();
    System.out.println("Array ordenado:");
    imprimirArray(array);
}
}

```

Saída do programa

```

Array original:
[64, 34, 25, 12, 22, 11, 90]

```

```

Passagem 1: [34, 25, 12, 22, 11, 64, 90]
Passagem 2: [25, 12, 22, 11, 34, 64, 90]

```

Passagem 3: [12, 22, 11, 25, 34, 64, 90]
Passagem 4: [12, 11, 22, 25, 34, 64, 90]
Passagem 5: [11, 12, 22, 25, 34, 64, 90]
Passagem 6: [11, 12, 22, 25, 34, 64, 90]

Array ordenado:
[11, 12, 22, 25, 34, 64, 90]

Eficiência

Caso	Comportamento
Melhor caso	Rápido — array já ordenado, só confirma
Caso médio	Lento — dobrar os dados quadruplica o tempo
Pior caso	Muito lento — array completamente invertido

- Faz **muitas comparações** mesmo quando o array está quase ordenado
- À medida que o número de dados aumenta, o tempo cresce muito rápido
- Para 100 elementos → ~10.000 operações; para 1.000 → ~1.000.000

Vantagens e Desvantagens

Vantagens

- Muito fácil de entender e implementar
- Funciona bem para arrays **pequenos**
- Estável (não altera a ordem de elementos iguais)

Desvantagens

- Lento com grandes volumes de dados
- Realiza muitas trocas desnecessárias
- Um dos algoritmos de ordenação mais ineficientes na prática

Shell Sort

O que é

Shell Sort é uma **evolução do Insertion Sort**. Em vez de comparar apenas elementos adjacentes, ele compara elementos **distantes entre si**, usando um valor chamado **gap (intervalo)**.

O gap começa grande (geralmente metade do tamanho do array) e vai sendo **reduzido pela metade** a cada rodada, até chegar em 1 — quando o algoritmo se comporta como um Insertion Sort comum, mas o array já está quase ordenado.

Como funciona

1. Define o gap inicial como $n / 2$
2. Compara e ordena elementos separados pelo gap
3. Reduz o gap pela metade: $gap = gap / 2$
4. Repete até $gap = 1$
5. Com gap 1, faz a ordenação final (array já está quase ordenado, então é rápido)

Exemplo passo a passo

Array inicial: [64, 34, 25, 12, 22, 11, 90] — tamanho 7, gap inicial = 3

Gap = 3 — compara posições (0,3), (1,4), (2,5), (3,6)...

```
[64, 34, 25, 12, 22, 11, 90]
  ↓           ↓
64 vs 12 → troca → [12, 34, 25, 64, 22, 11, 90]
                34 vs 22 → troca → [12, 22, 25, 64, 34, 11, 90]
                        25 vs 11 → troca → [12, 22, 11, 64, 34, 25, 90]
```

Resultado com gap 3: [12, 22, 11, 64, 34, 25, 90]

Gap = 1 — Insertion Sort normal, mas array já está quase ordenado

Resultado com gap 1: [11, 12, 22, 25, 34, 64, 90] 

Código Java

```
public class ShellSort {

    public static void shellSort(int[] array) {
        int n = array.length;

        // Define o intervalo inicial (gap) como metade do tamanho do
        array
        int gap = n / 2;

        while (gap > 0) {
            System.out.println("— Gap = " + gap + " —");

            // A partir do elemento na posição gap, compara com o
            elemento gap posições antes
            for (int i = gap; i < n; i++) {
```

```

        int temp = array[i];
        int j = i;

        // Desloca os elementos maiores que temp para frente
        while (j ≥ gap && array[j - gap] > temp) {
            array[j] = array[j - gap];
            j -= gap;
        }

        array[j] = temp;
    }

    imprimirArray(array);

    // Reduz o gap pela metade
    gap /= 2;
}

}

public static void imprimirArray(int[] array) {
    System.out.print("[");
    for (int i = 0; i < array.length; i++) {
        System.out.print(array[i]);
        if (i < array.length - 1) System.out.print(", ");
    }
    System.out.println("]");
}

public static void main(String[] args) {
    int[] array = {64, 34, 25, 12, 22, 11, 90};

    System.out.println("Array original:");
    imprimirArray(array);
    System.out.println();

    shellSort(array);

    System.out.println();
    System.out.println("Array ordenado:");
    imprimirArray(array);
}
}

```

Saída do programa

```

Array original:
[64, 34, 25, 12, 22, 11, 90]

```

```

— Gap = 3 —

```

```
[12, 22, 11, 64, 34, 25, 90]
```

```
— Gap = 1 —
```

```
[11, 12, 22, 25, 34, 64, 90]
```

Array ordenado:

```
[11, 12, 22, 25, 34, 64, 90]
```

Eficiência

Caso	Comportamento
Melhor caso	Rápido — cresce pouco mesmo com muitos dados
Caso médio	Moderado — bem melhor que Bubble Sort
Pior caso	Lento — mas ainda melhor que Bubble Sort puro

- **Muito mais rápido que o Bubble Sort** na prática
- O número de comparações cai drasticamente porque elementos distantes são movidos em menos passos
- Quando o gap chega a 1, o array já está quase ordenado, então a etapa final é rápida

Vantagens e Desvantagens

Vantagens

- Significativamente mais rápido que Bubble Sort e Insertion Sort para arrays grandes
- Implementação relativamente simples
- Não precisa de memória extra (ordena no próprio array — *in-place*)

Desvantagens

- Não é estável (pode alterar a ordem de elementos iguais)
- A eficiência depende da escolha da sequência de gaps
- Mais difícil de entender que o Bubble Sort

Comparação direta

Critério	Bubble Sort	Shell Sort
Velocidade média	Lento — tempo cresce muito com mais dados	Rápido — cresce pouco com mais dados
Facilidade de entender	Alta	Média

Critério	Bubble Sort	Shell Sort
Velocidade prática	Lento	Rápido
Estável	✅ Sim	❌ Não
Memória extra	Não precisa	Não precisa
Ideal para	Arrays pequenos	Arrays médios/grandes